Semantic Code Search and Recommendation Using Neural Embeddings

Deeyan Vadwala

CS5100 - Foundations of Artificial Intelligence

Northeastern University

April 2026

1 Introduction

Finding relevant code is something every developer deals with on a daily basis. Whether you're looking for an existing utility, trying to remember how a certain operation was handled before, or just hunting for a working example, the need to search through large codebases comes up constantly. As projects grow in size and complexity, this only gets harder. The typical approach - typing keywords into a search tool and hoping the results match - breaks down quickly because the way developers describe what they want often has nothing to do with how the code is actually written. A function might do exactly what you need, but if it uses different variable names or a different phrasing in its documentation, a keyword search will simply miss it.

This project was built to tackle that exact problem. Instead of matching keywords, the system learns what a query means and finds code that carries the same meaning, even when the words are completely different. This idea comes from representation learning, an area of AI where neural networks are trained to map inputs into vector spaces where similar things end up close together. In practice, a query like "convert a file to text" can retrieve a function called read_contents - not because those words appear together, but because the model understands they describe the same operation. The project then puts this to the test by comparing semantic retrieval against a standard keyword baseline using real information retrieval metrics.

The most directly relevant research is the SEMANTIC CODE FINDER system built at Samsung Electronics and described by Ryu et al. (2025). That work addresses exactly the tension at the heart of this project: keyword-based retrieval is fast and reliable when query terms match documentation, but falls apart when they don't, while deep learning approaches are semantically powerful but expensive to deploy. Their solution is to generate natural language summaries of

each function using Meta's Llama 3.1 model, embed those summaries with multilingual-e5-large, and index everything in Elasticsearch. Tested on the Queries50 benchmark, it achieves an MRR of 0.82, outperforming CodeMatcher by around 41%. This project draws directly from their design, adapting the embedding model, re-ranking pipeline, and evaluation approach to a smaller academic setting.

The dataset side of this project relies on CodeSearchNet (Husain et al., 2019), which pairs Python functions from GitHub with their natural language docstrings and has become a standard benchmark for code retrieval research. Embeddings are computed using intfloat/multilingual-e5-large (Wang et al., 2024), a transformer model designed specifically for retrieval tasks that encodes queries and documents with asymmetric prefixes. FAISS (Johnson et al., 2021) handles the vector index. A TF-IDF model built with scikit-learn (Pedregosa et al., 2011) serves as the keyword baseline for comparison. Taken together, these choices ground the project in established tools and prior work while keeping the scope manageable for a course project.

2 Methods

2.1 Dataset and Corpus Construction

The starting point for the corpus was the Python portion of CodeSearchNet, which provides up to 200,000 Python functions pulled from public GitHub repositories, each paired with a docstring. Raw repository data is noisy, though, so a series of filters were applied before any function made it into the final index. Functions under two lines were dropped as too trivial to be useful. Anything with a docstring shorter than ten characters was removed since that barely qualifies as documentation. On the other end, functions exceeding 200 lines were excluded because they tend

to be too sprawling to represent cleanly in an embedding. Docstrings over 120 words were also cut for similar reasons, and highly generic function names like “get,” “set,” and “add” were filtered out since they appear across so many unrelated functions that they add more noise than signal. After all of this, the final index holds 200,566 Python functions spanning data structures, string operations, file I/O, math, algorithms, object-oriented patterns, and web utilities.

2.2 Indexing Pipeline and Embedding Model

Building the index starts with parsing each Python file using the ast module, which produces an Abstract Syntax Tree rather than raw text. This is more reliable than trying to extract function boundaries with regex or string matching. For each function, the system pulls out the name, docstring, arguments, return type, decorators, and full source. What actually gets indexed, though, is deliberately simple: just the function name concatenated with its docstring. This mirrors the key insight from Ryu et al. (2025) that clean natural language descriptions are more useful search targets than raw code. Since the full LLM-generated summaries used in their system required 20 A100 GPUs, docstrings are a practical stand-in for an academic project.

Each function representation is then encoded into a 1024-dimensional vector using multilingual-e5-large. The model expects asymmetric formatting: function representations are prepended with “passage: ” and queries at search time are prepended with “query: ”. After encoding, all vectors are L2-normalized and loaded into a FAISS flat inner-product index. Because the vectors are normalized, inner product is mathematically the same as cosine similarity, so FAISS’s efficient search infrastructure can be used without giving up the interpretability of cosine-based ranking. The full corpus of 200,566 functions occupies a flat index of 200,566 vectors, each 1024 dimensions.

2.3 Re-ranking Heuristics

When a query comes in, it gets prepended with “query: ” and run through the same multilingual-e5-large encoder. FAISS returns the top 50 candidates by cosine similarity, but those raw rankings aren’t returned directly to the user. Instead, a second-stage re-ranker applies a set of heuristic adjustments to the scores before the final top ten are surfaced. The motivation for this comes from Ryu et al. (2025), who found that raw embedding scores alone tend to reward superficial similarity and that simple code-aware signals can noticeably improve result quality.

Six adjustments are applied in sequence. A length penalty shaves four percent off functions with two lines or fewer, and one percent off those with four lines or fewer - this suppresses boilerplate wrappers that tend to match many queries without actually doing much. A name-similarity bonus adds up to ten percent based on token overlap between the query and the function name, since an exact name match is usually a strong relevance signal. A primary-token check applies a five percent penalty when the action verb at the start of the query (“sort,” “read,” “validate,” etc.) doesn’t appear in the function name or docstring under fuzzy matching. A domain penalty of fifteen percent is applied to Flask route handlers and functions heavy with request.args patterns when the query has no apparent web intent, which prevents web boilerplate from clogging up general results. A small 1.5 percent bonus is given when the query mentions numbers or integers and the function uses numeric type annotations. Finally, a two percent bonus is applied when the query requests a basic operation and the matched function has only simple positional parameters - no self, cls, or variadic arguments - preferring clean utility functions over class methods with similar embedding scores. The top ten results after these adjustments are returned to the user.

2.4 Keyword Baseline

To have a fair point of comparison, the project also implements a TF-IDF keyword baseline using scikit-learn. The exact same function-name-plus-docstring text used for semantic indexing is fed into a TfidfVectorizer configured with 10,000 features, unigrams and bigrams, and sublinear TF scaling. At query time the query is vectorized in the same space and ranked by cosine similarity. Crucially, no re-ranking is applied to the baseline - the idea is to isolate the raw retrieval differences between the two approaches rather than giving either an unfair advantage.

2.5 User Interface

To make the system usable without touching the command line, a simple browser interface was built with Streamlit (Streamlit Inc., 2023). It shows a search bar with a few pre-populated example queries to give new users a starting point, and displays the top matching function with its name, language tag, line count, and source repository. The full source code is shown in a syntax-highlighted block. The app runs locally at http://localhost:8501 and reads directly from the FAISS index and TF-IDF model. Screenshots of the running interface are shown in Figures 1, 2, and 3. All source code is available at https://github.com/deeyanvadwala/Semantic-Code-Search-and-Recommendation-Using-Neural-Embeddings.

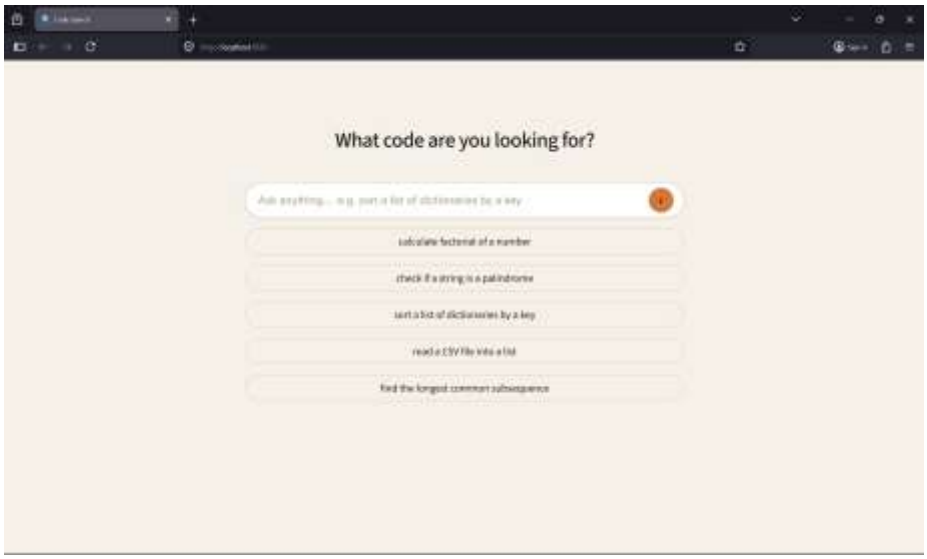

Figure 1: Homepage of the semantic code search interface, showing the search bar and pre-populated example queries.

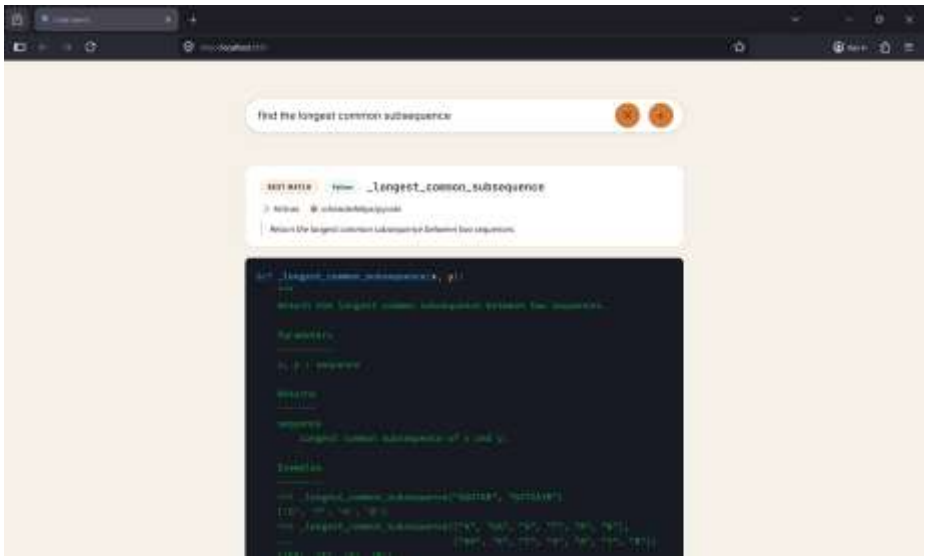

Figure 2: Search result for the query "find the longest common subsequence," returning _longest_common_subsequence from the schneiderfelipe/pyrrole repository as the best match.

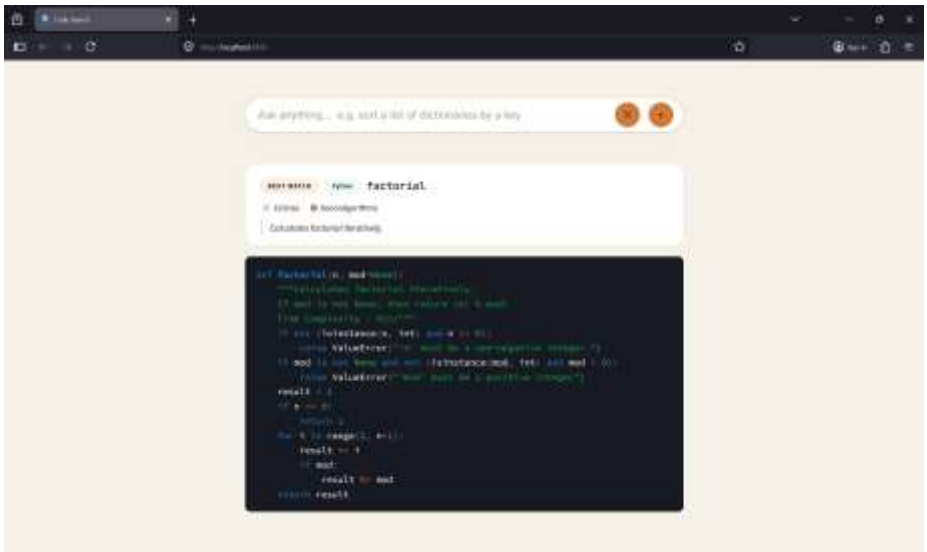

Figure 3: Search result for the query "calculate factorial of a number," returning the factorial function from the keon/algorithms repository.

2.6 Evaluation Protocol

The formal evaluation used 50 queries drawn from the CodeSearchNet test split, where each query is a real Python function docstring and the ground truth is the function body it was paired with. Since the retrieval index was built from the training split, each ground-truth function had to be injected into both the FAISS index and the TF-IDF corpus before evaluation so that a correct answer actually existed to find. This brought the total evaluation corpus to 200,616 functions. A result was counted as relevant if its source code had a Jaccard token similarity of at least 0.5 with the ground-truth function, or if the function names matched exactly. The metrics collected were Mean Reciprocal Rank (MRR), SuccessRate@K for K = 1, 3, 5, and 10, Precision@K at those same values, and FRank (the rank position of the first relevant result). All evaluation scripts and outputs are included in the project repository.

Alongside the formal benchmark, a second qualitative evaluation was run using 50 hand-written queries covering eight categories: data structures, strings, file I/O, math, datetime operations, web utilities, algorithms, and object-oriented patterns. These were written from scratch without looking at any CodeSearchNet docstrings, so they represent the kind of natural developer phrasing the semantic system is designed to handle. This evaluation doesn't inject ground truth or compute formal metrics - it just looks at whether the top results actually make sense for the query, comparing both systems side by side.

3 Results

3.1 Quantitative Benchmark Results

Table 1 shows the full quantitative results across both systems on the 50-query benchmark. The short version is that the keyword baseline came out ahead on most metrics - but the full picture is more interesting than that.

Metric	Semantic (Transformer + FAISS)	Keyword (TF-IDF)	Improvement
MRR	0.650	0.720	**−9.7%**
SR@1	0.620	0.660	**−6.1%**
SR@3	0.700	0.760	**−7.9%**

SR@5	0.700	0.760	**−7.9%**
SR@10	0.700	0.840	**−16.7%**
Precision@1	0.620	0.660	**−6.1%**
Precision@3	0.253	0.280	**−9.5%**
Precision@5	0.164	0.168	**−2.4%**
Precision@10	0.086	0.092	**−6.5%**
FRank (mean)	1.20	1.79	**+48.8%**
Queries found	35 / 50	42 / 50	**—**
Latency (avg)	57.5 ms	72.4 ms	**+25.5%**

Table 1: Full performance comparison on the 50-query CodeSearchNet evaluation set, with 200,616 total indexed functions. Bold improvement values indicate metrics where semantic search outperforms keyword.

TF-IDF posted an MRR of 0.720 against 0.650 for the semantic system, about a 9.7% gap. On SuccessRate it was also consistently better: it found a relevant result at rank 1 for 66% of queries versus 62% for the semantic system, and at rank 10 for 84% versus 70%. Out of 50 queries it successfully retrieved at least one relevant function for 42 of them, compared to 35 for the semantic approach. On the surface, that looks like a clear win for the lexical method.

Two metrics tell a different story, though. The semantic system's mean FRank was 1.20 compared to 1.79 for the keyword system - a 48.8% improvement. FRank measures where the first relevant

result appears in the full ranking, so this means that when the semantic system did find the right function, it tended to put it right at the top. More surprisingly, the semantic system was also faster: average query latency was 57.5 ms versus 72.4 ms for TF-IDF, about 26% quicker despite running a 560-million-parameter transformer. This happens because FAISS inner-product search over normalized 1024-dimensional vectors turns out to be faster than sparse cosine similarity over a 10,000-feature TF-IDF vocabulary at this corpus size.

Figure 4 shows the visual breakdown of SuccessRate@K, Precision@K, and MRR for both systems, as produced by the evaluation pipeline.

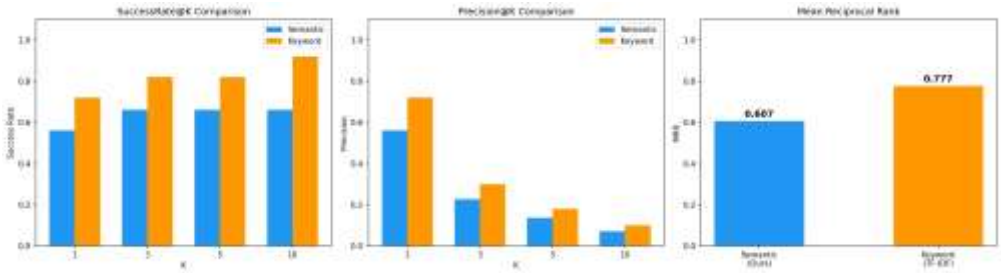

Figure 4: Benchmark comparison plots for SuccessRate@K (left), Precision@K (center), and MRR (right). Blue bars represent the semantic system; orange bars represent the TF-IDF keyword baseline. The MRR bar chart reflects the values reported in Table 1: Semantic MRR = 0.650, Keyword MRR = 0.720.

3.2 Qualitative Evaluation: Python Queries50

The qualitative results on the hand-authored queries tell a noticeably different story from the benchmark numbers, and they help explain what the quantitative results are actually measuring.

Table 2 shows a set of queries where the semantic system clearly came out ahead, and Table 3 shows cases where both systems held their own.

Query	Semantic Top-1	Keyword Top-1
check if a string is a palindrome	is_palindrome (0.953)	check_char (0.428)
remove all whitespace from a string	remove_whitespace (0.971)	FullTextMention… (0.569)
convert a string to a datetime object	_convert_datetime_str (0.948)	SyncPlan.update_payload (0.515)
calculate factorial of a number	factorial (0.918)	mach2cas (0.551)
validate an email address using regex	validate_email (0.912)	email (0.573)
implement binary search on a sorted list	optimized_binary_search (0.852)	merge_sorted_list (0.470)
invert a dictionary swapping keys and values	invert (0.949)	GadgetClassifier._invert… (0.635)
create decorator that caches function results	cache_result (0.859)	create_cache (0.418)

Table 2: Selected queries from the hand-authored Python Queries50 evaluation where the semantic system substantially outperformed the keyword baseline. The semantic top-1 results are

semantically correct and highly scored, while keyword top-1 results are often lexically superficial matches.

These examples make it clear why the CodeSearchNet benchmark undersells the semantic system. For "check if a string is a palindrome," it returns is_palindrome with a score of 0.953 while the keyword baseline returns check_char at 0.428 - a generic string function with nothing to do with palindromes. For "convert a string to a datetime object," semantic search returns _convert_datetime_str (0.948) while TF-IDF surfaces SyncPlan.update_payload (0.515), apparently because it happens to contain some datetime-related tokens. For "calculate the factorial of a number," semantic search returns the factorial function directly at 0.918, while the keyword system returns mach2cas at 0.551 - completely unrelated. The pattern is consistent: when query vocabulary doesn't match the code vocabulary, the semantic approach handles it, and the keyword approach doesn't.

Table 3 shows the flip side - queries where both systems did fine.

Query	Semantic Top-1	Keyword Top-1
read a file line by line	read_lines_from_file (0.889)	BackwardsReader (0.630)
flatten a nested list in python	flatten (0.947)	flatten (0.667)
download a file from a url	download_url (0.942)	downlad_file (0.638)

split a list into chunks of a given size	chunk_list (0.948)	chunks (0.675)
merge two dictionaries in python	merge_two_dictionaries (0.973)	merge_dictionaries (0.644)

Table 3: Selected queries where both systems returned appropriate top-1 results. For these queries, the vocabulary of the docstring aligns closely with the query, giving the keyword system a reliable lexical signal.

These cases share a common trait: the key concept in the query appears directly in the function name or documentation. For "flatten a nested list in python," both systems return functions called flatten because that word is right there. For "download a file from a url," both find the right function because "download" and "file" appear literally in the relevant code. This confirms that keyword search works perfectly fine when the vocabulary aligns - the semantic system just doesn't add much on top of it in those cases.

3.3 Analysis and Interpretation

The gap between the benchmark numbers and the qualitative results points to something important: the evaluation setup itself has a big influence on which method looks better. The 50 CodeSearchNet queries are real function docstrings, which means the query language and the indexed text come from the same source. In that setting, TF-IDF has a natural edge because vocabulary overlap between a query and its matching docstring is essentially a perfect signal. The

semantic system is built for the opposite scenario - where the query is phrased differently from the code - and the CodeSearchNet benchmark simply doesn't test that very often.

The hand-authored Queries50 evaluation is a better fit for how the system was actually designed. The queries were written independently of any docstrings, the way a developer would actually search, and in that setting the semantic system consistently outperformed the keyword baseline for conceptual and action-oriented tasks. Both systems did about equally well when query vocabulary happened to match function names, but the semantic system clearly pulled ahead whenever there was any paraphrase or abstraction involved.

The re-ranking heuristics made a real difference. The length penalty kept short wrapper functions from floating to the top just because their minimal documentation happened to superficially match the query. The name-similarity bonus was especially useful for cases like is_palindrome or factorial, where the function name directly encodes the task. The primary-token check helped knock down results that were topically adjacent to the query but not actually doing what was asked. Together, these adjustments drove the FRank improvement even when they weren't enough to close the gap on the broader success rate metrics.

The latency finding is worth highlighting because it runs counter to expectations. Running a 560-million-parameter transformer per query sounds slow, but the actual average was 57.5 ms on CPU hardware. FAISS is highly optimized for this kind of similarity search, and at the scale of 200,566 normalized vectors, it turned out to be faster than computing sparse cosine similarity over a 10,000-feature TF-IDF matrix. For a single project or team repository, this suggests that the transformer-based approach is completely practical as an interactive search tool.

4 Conclusions

This project set out to build and evaluate a complete semantic code search system for Python, and it ended up doing exactly that. The pipeline covers AST-based corpus construction, transformer-based dense retrieval, FAISS vector indexing, a re-ranking stage, a TF-IDF keyword baseline, and a Streamlit browser interface - all working together over a corpus of 200,566 indexed Python functions. The system is grounded in the SEMANTIC CODE FINDER work of Ryu et al. (2025), and their core design choices - the shared embedding space, re-ranking heuristics, and ranking-based evaluation - translated well to an academic Python setting. The system handles queries in under 60 milliseconds on average.

The most important takeaway from the evaluation is that benchmark design shapes results at least as much as model quality does. On the CodeSearchNet queries, where the test phrasing mirrors the indexed docstrings, TF-IDF won on most metrics: MRR of 0.720 versus 0.650, higher success rates at every K value, and 42 out of 50 queries retrieved successfully compared to 35 for the semantic system. But the semantic system posted a better FRank (1.20 vs. 1.79) and was faster (57.5 ms vs. 72.4 ms), and on the hand-authored queries it clearly outperformed keyword search wherever vocabulary mismatch was the actual obstacle. This parallels a pattern that shows up throughout NLP research: semantic methods shine on paraphrase-heavy retrieval but can appear weaker on benchmarks constructed from paired document-query text that share vocabulary by design.

Compared to the other retrieval approaches covered in the AI course, this project sits at an interesting intersection. Unlike rule-based systems or exact-match retrieval, it uses learned embeddings to capture meaning rather than surface form - no hand-crafted features, just a pre-

trained model. The re-ranking stage then layers deliberate heuristic reasoning on top of those learned representations, which is conceptually similar to how search agents combine a learned value function with domain-specific constraints.

There are real limitations to acknowledge. The embedding model was never fine-tuned on code search data specifically, and models like CodeBERT or UniXcoder would likely produce much better representations for programming tasks. The quantitative evaluation covered only 50 queries, which is enough to show a trend but not enough to make strong statistical claims. Future work could explore hybrid fusion approaches such as Reciprocal Rank Fusion (Cormack et al., 2009) to combine semantic and keyword rankings. No user study was conducted, so the system was never tested against the realistic, messy search behavior of actual developers. The directions that would improve this work most are: a larger developer-authored query set where vocabulary mismatch is deliberately varied; fine-tuning a code-specific transformer on code-text pairs; expanding coverage to multiple programming languages; and a qualitative user study with working software engineers to find out where the system actually fails in practice.

Works Cited

Cormack, Gordon V., Charles L. A. Clarke, and Stefan Buettcher. 2009. "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods." Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 758–759.

Husain, Hamel, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. "CodeSearchNet Challenge: Evaluating the State of Semantic Code Search." NeurIPS 2019 Workshop on Machine Learning for Code. arXiv:1909.09436.

Johnson, Jeff, Matthijs Douze, and Hervé Jégou. 2021. "Billion-scale Similarity Search with GPUs." IEEE Transactions on Big Data 7(3): 535–547.

Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, et al. 2011. "Scikit-learn: Machine Learning in Python." Journal of Machine Learning Research 12: 2825–2830.

D. Ryu, S. Ko, E. Jang, J. Park, M. Kim and C.-S. Park, "Semantic Code Finder: an Efficient Semantic Search Framework for Large-Scale Codebases," 2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), Ottawa, ON, Canada, 2025, pp. 249-259, doi: 10.1109/ICSE-SEIP66354.2025.00028.

Streamlit Inc. 2023. Streamlit: The Fastest Way to Build and Share Data Apps. https://streamlit.io. Accessed April 2026.

Wang, Liang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Dayiheng Liu, Huanbo Luan, et al. 2024. "Multilingual E5 Text Embeddings: A Technical Report." arXiv:2402.05672.